

AURIX: Backend & Infrastructure Engineer Roadmap (Node.js)

Assignee: Bhavya Gupta **Role:** Backend & Infrastructure Engineer (The "Traffic Cop", "Vault", and Nervous System) **Primary Focus:** Layer 2 (Backend API & Storage Gateway) & RAG Vector Infrastructure

This roadmap outlines your journey to build the backend infrastructure for the **AURIX** platform from scratch to production, specifically tailored for a **Node.js** ecosystem as requested.



Phase 1: Infrastructure Provisioning & Foundation

Your first goal is to set up the foundational cloud infrastructure and database schemas.

1.1 Cloud Services Setup

- ☐ **Supabase (Free Tier):** Create a new Supabase project. This will provide PostgreSQL, Supabase Storage (S3-compatible), and Supabase Auth.
- ☐ **Upstash (Serverless Redis):** Create a Redis database on Upstash for task queuing and rate limiting.

1.2 Database Schema Implementation

Execute the SQL (or use Prisma/Drizzle ORM if preferred) to create the core tables mapped in the `database-er-diagram`:

- ☐ **USERS:** Managed largely by Supabase Auth (linking UUIDs).
- ☐ **PROJECTS:** `id`, `user_id` (FK), `name`, `repository_url`, `created_at`.
- ☐ **SCANS:** `id`, `project_id` (FK), `status` (PENDING, SCANNING, COMPLETED, FAILED), `storage_path`, `total_findings`, `verified_findings`, `timestamps`.
- ☐ **VERIFIED_VULNERABILITIES:** `id`, `scan_id` (FK), `file_path`, `line_number`, `severity`, `vulnerability_type`, `ai_confidence_score`, `logic_reasoning`, `wargame_status`, `poc_script`, `patch_code`, `is_resolved`.

1.3 Authentication ("The Vault")

- ☐ Configure **Supabase Auth** (Email/Password or GitHub OAuth).
- ☐ Implement middleware in Node.js to validate secure session tokens (JWTs).
- ☐ Ensure users can *only* query and view their own scan reports and projects (Row Level Security in Supabase or Backend validation).

Phase 2: Core API Gateway & Unblocking Teammates

CRITICAL BLOCKER ALERT: You block Bhumika (Frontend) and Divyanshi (VS Code). Your immediate priority is to deploy **Mock Endpoints** so they can start building.

2.1 The "Mock" Unblocker

- ☐ Initialize a Node.js Express server.
- ☐ Create mock endpoints for `/api/scans/github` and `/api/scans/upload` that return fake `scan_ids` and 202 Accepted.
- ☐ Create a mock `/api/scans/{scan_id}` that returns

hardcoded JSON vulnerabilities.

- ☐ Deploy this mock server immediately (e.g., Render, Railway, or Vercel).

2.2 Core Ingestion Routes

Once teammates are unblocked, replace mocks with real logic:

- ☐ **POST /api/scans/github**: Save project to DB, set status to `PENDING`, push the GitHub URL to the Redis Queue, return `202 Accepted`.
- ☐ **POST /api/scans/upload**: Use `multer` to handle `multipart/form-data`. Safely stream `.zip` files (~10MB limit) directly to Supabase Storage. Create DB record, push storage link to Redis, return `202 Accepted`.
- ☐ **GET /api/scans/{scan_id}**: Look up scan in DB. Return current status (e.g., `SCANNING`) or the final `VERIFIED_VULNERABILITIES` if `COMPLETED`.

⚠ **Asynchronous Processing is Mandatory**: Never await the AI scan in your HTTP POST request. It takes minutes and will timeout. Return `202 Accepted` immediately after pushing to Redis.

Phase 3: The AI Upgrade - RAG Vector Infrastructure

You are responsible for the infrastructure that allows the AI agents to query OWASP guidelines and past CVEs to fix bugs accurately without hallucinating.

3.1 Vector Database Setup

- ☐ Enable the `pgvector` extension in your Supabase PostgreSQL database (`CREATE EXTENSION vector;`).

- ☐ Design and create tables to store high-dimensional vector embeddings (e.g., a `knowledge_base` table with a `vector` column).

3.2 Data Seeding Script (`seed_rag.js`)

- ☐ Write a standalone Node.js script to ingest raw security docs (OWASP Cheat Sheets, Vulnerability PoC templates for Python/JS, Patch Code Templates).
- ☐ Use an embedding model API (like OpenAI `text-embedding-3-small` or HuggingFace API) to convert these markdown chunks into vector embeddings.
- ☐ Push the text and their embeddings into the Supabase vector table.

3.3 Similarity Search API (Threat Intel Endpoint)

- ☐ Create an internal API endpoint (or Supabase RPC function) for Divyansh's LangGraph node to call.
 - ☐ **Input:** Search query (e.g., `{"query": "python.lang.security.deserialization.pickle.avoid-pickle python"}`).
 - ☐ **Processing:** Convert query to an embedding, perform cosine similarity search against `pgvector`.
 - ☐ **Output:** Return a JSON array of the Top 2 or 3 most relevant markdown text chunks.
 - ☐ **Constraint:** Must return results in **under 500ms**.
-

Phase 4: Orchestration, Security, & Optimization

Connect the final pieces and ensure the system is secure and cost-effective.

4.1 Internal Webhook (AI Worker to Backend)

- ☐ Create an internal `POST` route (e.g., `/api/internal/webhook/scan-complete`).
- ☐ Divyansh's AI Worker will call this when a scan finishes, sending the final verified bugs payload.
- ☐ Update the `SCANS` table status to `COMPLETED` or `FAILED` and populate the `VERIFIED_VULNERABILITIES` table.

4.2 Rate Limiting

- ☐ Implement Redis-backed API Rate Limiting (using libraries like `express-rate-limit` + `rate-limit-redis`).
- ☐ Restrict users to a maximum number of scans per hour/day to protect computationally expensive AI resources. Return `429 Too Many Requests` if exceeded.

4.3 Automated Data Sanitization

- ☐ Implement a cron job (using `node-cron` or a serverless function schedule) or Supabase Storage TTL.
- ☐ Automatically delete the `.zip` payload from Supabase Storage once a scan is marked `COMPLETED` or `FAILED` to prevent massive storage bloat and protect user source code privacy.

🚫 "Do Not Miss" Constraints Checklist

- ☐ **Auth Handshake:** Frontend has exact routes for login/signup; all GET/POST routes are secured.
- ☐ **Robust Zip Handling:** Streams are handled without crashing your lightweight Node.js server.
- ☐ **Data Privacy:** Source code (`.zip`) is deleted post-scan. Only vulnerability metadata is kept.
- ☐ **Compute Protection:** Redis rate limiter is active and prevents

queue flooding.

- ☐ **RAG Latency:** Vector retrieval happens in < 500ms using high-quality embeddings.